GOLD: Flipped Friday #3 (Fri Oct 9)

Flipped Friday #3

Labs Next Week

- Labs will still be held next week, but due to fall break, attendance will not be taken

Participation

- **iClicker:** participation is based on the polls only.
- **Ed challenges**: the test case scores are just for practice; we never include these in the class participation.

Working with Files

On Wednesday you watched files get read and written.

Today you write the two programs that an exam might ask for, from scratch: a tool that reads a whole file, and a program that reads one file and writes another. UTA staff are in the aisles.

- Exercise1: **Read It All** (~12 min)
- Whole-room: review and polls
- Exercise 2: **Orders In, Report Out** (~15 min): read a CSV file
- Review, closing, two polls.
- Finished early? **Append to the Log**, the optional last slide.

Each challenge is a small workspace with its data file already in it (`steps.txt`, `orders.csv`).

Run runs `Main`; **Test** runs the tests. Classes in files need `import java.io.*;` at the top; the starter code already has the imports.

Exercise: Read It All

Our first exercise will be to practice writing out Wednesday's file-reading ceremony as a tool. **`FileTools.java` has one method to finish**, and `Main.java` is given to you already complete. The `main` method totals `steps.txt`, containing one day's step count per line, where a blank line is a day with no record.

Finish `FileTools.readAllLines(String path)`. Open the file, read it line by line until `readLine()` gives you `null`, put every line in an `ArrayList<String>`, close the file, return the list.

- Every line goes in, in order, including a blank one: a blank line is `""`, an empty String, not the end of the file.
- An empty file gives an empty list.
- If the file cannot be opened, let the `IOException` out:
  - the method already says `throws IOException`,
  - and `Main` is where that decision belongs.

As given, **Run** crashes on the first line of `main`: the method returns `null`. When it works, Run prints `Days: 5`, `Total: 37050` and `Best: 12300`.

Press **Test** to run six unit tests; they write their own small files, so they do not depend on `steps.txt`.

Whole-Room: Common Wrong Answers

Here are two implementations we might see.

The first:

```
void main() throws IOException {
    Files.writeString(Path.of("steps.txt"), "8200\n10450\n\n6100\n12300\n");   // give

    ArrayList<String> lines = readAllLines("steps.txt");
    IO.println(lines.size() + " lines: " + lines);
}

ArrayList<String> readAllLines(String path) throws IOException {
    ArrayList<String> lines = new ArrayList<>();
    BufferedReader br = new BufferedReader(new FileReader(path));
    while (br.readLine() != null) {
        lines.add(br.readLine());
    }
    br.close();
    return lines;
}
```

Pause and predict (poll time)

Q. `steps.txt` is the burst's file: `8200`, `10450`, a blank line, `6100`, `12300`. The helper above calls `readLine()` in the loop condition **and** in the loop body. What is in the list it returns?

- **A.** All five lines, in order
- **B.** `10450`, `6100`, `null`: every other line, then a `null`
- **C.** `8200`, `10450`, `6100`, `12300`: the blank line is skipped
- **D.** Nothing: it crashes with a `NullPointerException` inside the loop

The second:

```
void main() throws IOException {
    Files.writeString(Path.of("steps.txt"), "8200\n10450\n\n6100\n12300\n");   // give

    ArrayList<String> lines = readAllLines("steps.txt");
    IO.println(lines.size() + " lines: " + lines);
}

ArrayList<String> readAllLines(String path) throws IOException {
    ArrayList<String> lines = new ArrayList<>();
    BufferedReader br = new BufferedReader(new FileReader(path));
    String line = br.readLine();
    while (line != null && !line.isEmpty()) {   // stops at the first blank line
        lines.add(line);
        line = br.readLine();
    }
    br.close();
    return lines;
}
```

Run it: two lines, not five.

A blank line is `""`, an empty String; only the end of the file is `null`, so the loop tests `line != null` and nothing else.

Pause and predict (poll time)

Q. A different helper forgets the last step: it never calls `br.close()`. The list it returns is complete. What else is true?

- **A.** The lines it read are lost when the method returns, as with a writer
- **B.** The program crashes when it ends with a file still open
- **C.** Nothing visible: the data was read. The program holds the file open until it ends
- **D.** Compile error: a `BufferedReader` must be closed before the method returns

Build Your Own Tool: readAllLines

Warning: this only works with files that can fit in memory all at once

Build your own tool: `readAllLines`

The ceremony of reading from a file never changes:

- read each line and
- do something
- close it

Let's write it **once**, as a helper, and reuse it forever.

- We can read every line into an ArrayList

- In practice, files may be too large to read all at once
    - Then you process them line-by-line on the fly

```
void main() throws IOException {
    salesFile();                                    // given

    ArrayList<String> lines = readAllLines("sales.txt");

    double total = 0, max = 0;
    for (String s : lines) {
        double sale = Double.parseDouble(s);
        total += sale;
        if (sale > max) {
            max = sale;
        }
    }
    IO.println("Sales: " + lines.size());
    IO.println(String.format("Total: $%.2f", total));
    IO.println(String.format("Biggest sale: $%.2f", max));
}

ArrayList<String> readAllLines(String path) throws IOException {
    ArrayList<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String line = br.readLine();
        while (line != null) {
            lines.add(line);
            line = br.readLine();
        }
    }
    return lines;
}

// Given. The register wrote this file during the day; the slide writes it so that Run
void salesFile() throws IOException {
    PrintWriter pw = new PrintWriter(new FileOutputStream("sales.txt"));
    pw.println("5.49");
    pw.println("12.98");
    pw.println("7.25");
    pw.println("5.49");
    pw.println("21.47");
    pw.close();
}
```

```
Sales: 5
Total: $52.68
Biggest sale: $21.47
```

Now every file program can start the same way:

```
ArrayList<String> lines = readAllLines("sales.txt");
```

and from there it is list work.

CSV Files

The format is a contract

The cash register could also log *what* was sold, not just the price.

This means multiple pieces of data per order. If we want to stick to one line per order, we need to pick a format for the data.

A common format called **comma-separated values (CSV)** stores a table of information in plain text:

- one object (order) per line
- the fields (pieces of information) are written with a comma between them
- Example: `item,quantity,price`

```
BURGER,2,5.49
FRIES,3,2.75
SHAKE,1,4.95
```

We still need to tell the reader what format to expect (here String, then int, then double) so that the reading program knows how to parse the fields into the correct types.

Now let's try a challenge where we write this code ourselves.

Exercise: Orders In, Report Out

The campus store logs one order per line in `orders.csv`:

`ITEM,quantity,price`, as in `NOTEBOOK,4,3.25`.

The `orders.csv` file is in your workspace, and so is `FileTools.java`, the file-reading tool from the first exercise.

Finish `Main.java`: read the file, compute some stuff, and write a report. This is the shape of a typical exam question.

1. **Read: This part is given to you.** The first lines of `main` already call `FileTools.readAllLines("orders.csv")` and, if the file cannot be opened, print `Cannot open orders.csv:` followed by the exception's message and stop.

2. **Compute.** For each line, `split(",")` gives `{ITEM, quantity, price}`. The revenue of a line is quantity times price. You want the number of orders, the total revenue, and the item of the line with the largest revenue.

3. **Write.** `report.txt`, exactly three lines, for the data as given:

```
Orders: 4
Revenue: $64.49
Biggest: BACKPACK
```

Hints: Use `String.format("%.2f", ...)` for the revenue. Open the writer once, write, close it (or let try-with-resources close it): a file you did not close is a file you may not have written.

Press **Test** to run five tests.

Append or Overwrite?

Appending vs Overwriting is one key bit of syntax to remember; you don't want to choose the wrong one.

Append, or overwrite?

Opening a file for writing **erases it**, unless you say otherwise:

```
new FileOutputStream("log.txt")          // OVERWRITE: the file starts empty
new FileOutputStream("log.txt", true)    // APPEND: keep the contents, add to the end
```

```
void main() throws IOException {
    PrintWriter a = new PrintWriter(new FileOutputStream("log.txt"));          // OVER
    a.println("Monday: 5 sales");
    a.close();

    PrintWriter b = new PrintWriter(new FileOutputStream("log.txt", true));    // APPE
    b.println("Tuesday: 7 sales");
    b.close();
    IO.println("after the append:");
    showFile("log.txt");                                                       // give

    PrintWriter c = new PrintWriter(new FileOutputStream("log.txt"));          // OVER
    c.println("Wednesday: 6 sales");
    c.close();
    IO.println("after the overwrite:");
    showFile("log.txt");
}

// Reads a file back and prints it.
void showFile(String path) throws IOException {
    BufferedReader br = new BufferedReader(new FileReader(path));
    String line = br.readLine();
    while (line != null) {
        IO.println(line);
        line = br.readLine();
    }
    br.close();
}
```

```
after the append:
Monday: 5 sales
Tuesday: 7 sales
after the overwrite:
Wednesday: 6 sales
```

The register uses append: each day adds to the log.

A report uses overwrite: you want *this* report, not every report ever.

Closing

Review and Closing

A file is just characters.

The **format** (what the columns mean, in what order) is a promise between the program that writes and the program that reads.

If you break the promise then your data isn't readable (at least not without a lot of human effort).

Finished early? **Exercise (optional): Append to the Log**: write a program whose file remembers across runs.

Two last questions. Consider this possible answer to the second exercise:

```
for (String line : orders) {
    String[] parts = line.split(",");
    PrintWriter pw = new PrintWriter(new FileOutputStream("report.txt"));   // opened
    pw.println(parts[0] + ": " + Integer.parseInt(parts[1]) * Double.parseDouble(parts
    pw.close();
}
```

Pause and predict (poll time)

Q. Your friend's program opens the `PrintWriter` **inside** the loop, writes one line per order, and closes it each time. `orders.csv` has four orders. What is in `report.txt` at the end?

- **A.** All four lines, one per order
- **B.** The first line only
- **C.** The last line only: `STICKERS: 4.5`
- **D.** Nothing: 0 bytes, the file was never closed properly

Pause and predict (poll time)

Q. The register must **keep** yesterday's lines in `log.txt` and add today's at the end. Which opening line does that?

- **A.** `new PrintWriter(new FileOutputStream("log.txt"))`
- **B.** `new PrintWriter(new FileOutputStream("log.txt", true))`
- **C.** `new PrintWriter("log.txt", true)`

- **D.** `new PrintWriter(new FileOutputStream(new File("log.txt", true)))`

Exercise 3 (optional): Append to the Log

Optional, if you finish early. Write a visitor log that remembers data across program runs.

Every run of `Main` adds one line to `log.txt`, `Visit 1`, then `Visit 2`, and so on, and then prints the whole log. The number is one more than the lines already in the file; on the very first run there is no file yet, so the count starts at 0 (catch the `FileNotFoundException`, or check before you open).

The one idea this is about: `new FileOutputStream("log.txt", true)`. Without the `true`, every run would erase yesterday.

Run a few times and watch the log grow. Press **Test** to run two tests; they delete `log.txt` first and run your program three times.